

Optimizing Virtual Functions Deployment in Multi-UAV IoT Networks

Athena Forghani¹, Kwan-Wu Chin¹, and Montserrat Ros¹, *Senior Member, IEEE*

Abstract—In Internet of Things (IoT) networks, unmanned aerial vehicles (UAVs) play a critical role as mobile nodes that can be deployed to carry out data collection and computation. In this respect, this article considers an operator that deploys UAVs to satisfy requests from IoT applications that require virtual network functions (VNFs) that may communicate with one another to be executed at different geographical locations. To this end, this article formulates a novel mixed-integer linear program (MILP) to determine the optimal assignments of UAVs and VNFs over a planning horizon that maximizes a given performance metric, e.g., revenue. It also outlines a heuristic method named MPopLoc that chooses requests according to popular requested locations and traveling cost of UAVs. The results show that MPopLoc achieved approximately 95.14% of the optimal result.

Index Terms—Algorithm, containers, drone, network design, optimization, service, virtual machines.

I. INTRODUCTION

UNMANNED aerial vehicles (UAVs) or drones are ideal for providing services such as data collection in remote regions or not easily accessible areas. They are a key component of Internet of Things (IoT), where they can be used to monitor infrastructure such as power lines [1], and play a critical role in maritime environment [2]. In general, they can be used to improve the performance or extend the coverage of existing communication networks and have wide ranging applications in various industries [3], [4]. Some examples include businesses that offer UAVs/drones as a service in industries, such as cinematography [5] and real estate [6], where UAVs are used for aerial photography and videography. Further, these businesses can leverage virtualization technologies on UAVs to support various applications or services. For example, a UAV can run a Docker container [7], which facilitates the execution of different services or applications on a single UAV. This leads to more efficient uses of UAV resources, as well as easier management and deployment of different services or applications required by customers. Additionally, because Docker containers are lightweight and have low overhead, they can be scaled up or down easily depending on the resource demand of services or applications [8].

Manuscript received 24 December 2023; revised 4 February 2024; accepted 22 February 2024. Date of publication 29 February 2024; date of current version 23 May 2024. (Corresponding author: Athena Forghani.)

The authors are with the School of Electrical, Computer and Telecommunications Engineering, University of Wollongong, Wollongong, NSW 2522, Australia (e-mail: af149@uowmail.edu.au; kwanwu@uow.edu.au; montse@uow.edu.au).

Digital Object Identifier 10.1109/IIOT.2024.3371468

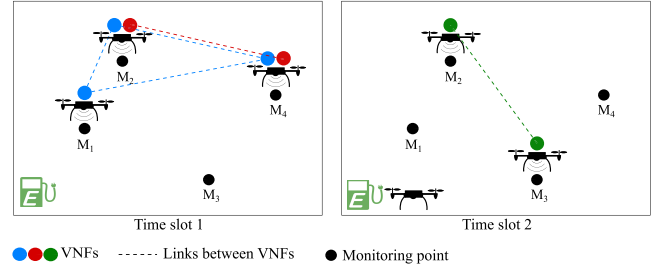


Fig. 1. Example of servicing request $r_1 = \{M_1, M_2, M_4\}$, $r_2 = \{M_2, M_3\}$, and $r_3 = \{M_2, M_4\}$ using three UAVs in two time slots.

In this respect, this article considers a *novel* problem faced by an operator of a fleet of UAVs that support virtualization. Specifically, given a set of service requests, the operator aims to maximize its reward by servicing as many requests as possible by deploying UAVs to run virtual network functions (VNFs) over a given time horizon. Note that each request contains VNF(s) to be run at various locations by UAV(s) and requires the operator to allocate resources such as computation and communication.

Fig. 1 provides an example of the said problem. There are three UAVs and also the following locations: M_1 , M_2 , M_3 , and M_4 . Further, there are three requests: $r_1 = \{M_1, M_2, M_4\}$, $r_2 = \{M_2, M_3\}$, and $r_3 = \{M_2, M_4\}$. Each request requires a VNF to be run at the stated locations, and VNFs communicate with one another. The shown UAVs can service all the requests in two time slots. Specifically, in time slot 1, UAVs service request r_1 and r_3 . To fulfill request r_2 , the UAV located at M_4 flies to M_3 in time slot 2, while the UAV at location M_2 continues to hover at the same location to service a VNF and to support ongoing communication with the VNF/UAV at location M_3 . Observe that the third UAV is not required to service request r_2 . Hence, it is able to fly to a base point to replace its battery or to receive a full charge.

The aforementioned problem is very challenging. First, the available UAVs, their resources and locations are a function of past serviced requests; i.e., the combination of requests serviced in prior time slots affect future requests serviced by an operator. Second, a special case of the said problem is the NP-hard cost-constrained traveling salesman problem (CCTSP) (see Section VI).

To this end, this article presents the following contributions.

- 1) It presents the first mixed-integer linear program (MILP) for the problem at hand, where it can be used

TABLE I
COMPARISON OF RELATED WORKS

Area	Prior works	Node deployment	Network construction	VNFs placement	Multiple time slots	Charging
<i>Uncontrolled mobility</i>	[11] [12]	✗	✗	✗	✗	✗
	[13] [14] [15]	✗	✗	✓	✗	✗
<i>Controlled mobility</i>	[16]	✓	✗	✗	✓	✓
	[17], [18]	✓	✓	✓	✗	✗
	This work	✓	✓	✓	✓	✓

to determine the deployment of UAVs to requested locations and the assignment of VNFs to UAVs to service requests within a given time horizon in order to optimize an operator's reward, such as revenue.

- 2) It presents two heuristic solutions called MPopLoc and LPopLoc; note, MILP can only be used on small problem instances. MPopLoc chooses requests with the most popular locations and UAVs with minimal traveling cost. However, LPopLoc prioritizes requests with the least popular locations.
- 3) It contains the first study of the problem, MILP, MPopLoc, and LPopLoc. The results show that the number of serviced requests is affected by different factors, including the number of UAVs, computation capacity and workload, battery capacity, area size, number of time slots, communication demand, the number of VNFs in each request, the number of requests, and revenue.

Next, Section II reviews prior works. Then, Sections III and IV, respectively, formalize the system and formulate the problem. Following this, Section V provides an outline of the proposed heuristic solutions. Section VI analyzes the said heuristic solutions and problem. Simulation results are presented and discussed in Section VII. Finally, Section VIII concludes this article.

II. RELATED WORKS

Many works have investigated virtualization in sensor-cloud infrastructures and wireless sensor networks (WSNs) to achieve certain objectives. They aim to embed virtual functions onto physical sensor nodes to satisfy requests. Some works, such as [9] and [10], consider fixed sensor nodes; i.e., they assume an *existing* network infrastructure. In contrast, this article considers network construction in each time slot.

A related research area is virtualization on mobile sensor nodes. Prior works can be categorized into *uncontrolled* and *controlled* scenarios. In the *uncontrolled* category, sensor nodes move as per some stochastic process. For example, Roy et al. [11] created virtual sensors on mobile devices to service requests, allowing a cloud provider to maximize profit by reallocating physical sensor nodes when one stops working or leaves a region of interest. Reference [12] explores a pricing scheme for cloud providers, mobile device owners, and sensor owners. Specifically, it considers multiple stakeholders and aims to determine an optimal pricing strategy that balances their interests and profits. The authors highlight the impact of device mobility on the duration of services provided by sensor nodes, which in turn affects the profit gained by sensor and

device owners. The work in [13] focuses on the migration of VNFs between mobile nodes, aiming to reduce frequent migrations caused by unpredictable node mobility. Reference [14] adapts to the dynamic nature of the network topology as vehicles move and change positions, ensuring connectivity between vehicles. The authors prioritize requests based on various attributes, such as preprocessing degree, service type, resource consumption, waiting time, and lifecycle. Finally, in [15], given a formation or network of UAVs, the goal is to embed VNFs onto a mobile sensor network of aircraft to service virtual network requests. In contrast to the above works, this article does not consider a fixed or given topology.

The second category of works, namely, *controlled* mobile sensor nodes, allow an operator to jointly optimize the movement/placement of nodes to facilitate virtual function execution. For example, in [16], the aim is to load VNFs onto UAVs, and design the trajectory of UAVs to run these VNFs at a set of locations. Note, these VNFs do not communicate with one another. Another example is [17], which aims to place the minimum number of UAVs that satisfy one or more requests or service function chains. The authors aim to create a UAV network to support VNFs. Finally, Wu et al. [18] constructed a network of UAVs to maximize the minimum amount of computed data between source-destination pairs. The problem at hand is to determine the placement of UAVs, routing and allocation of VNFs on each UAV.

In conclusion, there are no works that consider network construction, virtual network embedding and charging of UAVs jointly over time. Table I compares works with respect to controllability, node deployment, network construction, virtual network embedding, the number of time slots, and charging UAVs. This article considers optimizing the placement of UAVs and VNFs over time. Further, unlike works such as [16], it considers UAVs or/and VNFs that communicate with one another. Although [17] and [18] have network construction, they consider one time slot. Further, they do not recharge UAVs. Moreover, the work in [18] does not consider requests. Unlike works in the *uncontrolled* category, e.g., [11], in this work, the placement of nodes/UAVs can be controlled by an operator.

III. SYSTEM MODEL

Table II summarizes key notations. Let the set of geographical points/locations be $\mathcal{M} = \{0, 1, 2, \dots, M\}$. This article uses i and j to denote a geographical point/location in $\mathcal{M}$. Note, index 0 corresponds to the base point, and indices $1, 2, \dots, M$ refer to monitoring points.

TABLE II
COMMON NOTATIONS

Notations	Definitions
Sets	
$\mathcal{M}$	Set of geographical points/locations
$\mathcal{U}$	Set of quad-rotor UAVs
$\mathcal{R}$	Set of VNF requests
$\mathbf{r}_k$	Set of VNFs in the k -th request
$\mathcal{D}$	Set of traveling paths between locations
$\mathcal{K}^t$	Set of communication links between UAVs
$\mathcal{Z}_k$	Set of virtual links between VNFs in request $\mathbf{r}_k$
$\mathcal{N}_u^t$	Set of neighbors of UAV u in time slot t
Constants	
$g(i,j)$	Channel gain between location i and j
$C(i,j)$	Link capacity between location i and j
$q_{\alpha_{ki}}$	CPU frequency of VNF α_{ki}
$\Psi_{\alpha_{ki}}$	Computation workload of VNF α_{ki}
$C(\alpha_{ki}, \beta_{kj})$	Communication demand of virtual link $(\alpha_{ki}, \beta_{kj})$
W_k	Reward of request $\mathbf{r}_k$
Variables	
E_u^t	Energy level of UAV u in time slot t
ω_k^t	Binary variable indicating whether request $\mathbf{r}_k$ is serviced in time slot t
$b_u^t(\alpha_{ki})$	Binary variable indicating whether VNF α_{ki} is embedded on UAV u in time slot t
$f_{(u,v)}^{(\alpha_{ki}, \beta_{kj}), t}$	Flow rate when virtual link $(\alpha_{ki}, \beta_{kj})$ is embedded onto communication link (u, v) in time slot t
δ	The total energy that each UAV consumes to process VNF(s) in each time slot
θ_u^t	Binary variable indicating whether UAV u is active in time slot t
$\mu_u^t(i)$	Binary variable indicating whether UAV u is located at location i at time slot t
$\pi_u^{t,t+1}(i, j)$	Binary variable indicating whether UAV u travels over path $(i, j) \in \mathcal{D}$ between time slot t and $t + 1$

Let $\mathcal{G}(\mathcal{M}, \mathcal{D})$ be a graph, with $\mathcal{D}$ representing the set of traveling paths used by UAVs. Specifically, let a traveling path $(i, j) \in \mathcal{D}$ denote the flying path between locations i and j . Each location i has coordinate (x_i, y_i) . The Euclidean distance of path (i, j) is

$$d_{(i,j)} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}. \quad (1)$$

Time is discrete and indexed by t . Optimization is carried out over a planning horizon with T time slots, denoted as $\mathcal{T} = \{1, 2, \dots, T\}$. There is a set of quad-rotor UAVs, denoted as $\mathcal{U} = \{1, 2, \dots, U\}$. These UAVs are indexed as u or v . Each UAV has the following resources: 1) energy; 2) computation; and 3) communication bandwidth. Let $E_{\max}$ and $\Psi_{\max}$ be the battery size and computation capacity of UAVs, respectively. In each time slot t , when UAVs are deployed to a location in $\mathcal{M}$, they form a graph, which is modeled as $\mathcal{G}^t(\mathcal{U}^t, \mathcal{L}^t, \mathcal{K}^t)$; the set $\mathcal{U}^t \subset \mathcal{U}$ represents the deployed UAVs at time slot t , the set $\mathcal{L}^t = \{\mathcal{L}_u^t\}_{u \in \mathcal{U}} \subset \mathcal{M}$ represents the location of deployed UAVs at time slot t , and the set $\mathcal{K}^t$ denotes communication links, where $(u, v) \in \mathcal{K}^t$ represents the channel between UAV u and v . Define $C_{(i,j)}$ as the link capacity between locations i and j ; note this capacity is nonzero only if UAVs u and v are, respectively, placed at locations i and j . The set $\mathcal{N}_u^t \subset \mathcal{U}^t$ represents the neighbors of UAV u in graph $\mathcal{G}^t(\mathcal{U}^t, \mathcal{K}^t)$.

A. VNF Requests

Denote the set of VNF requests as $\mathcal{R} = \{\mathbf{r}_1, \mathbf{r}_2, \dots, \mathbf{r}_S\}$. Let W_k be how much request $\mathbf{r}_k$ pays if it is serviced. Each request $\mathbf{r}_k$ has $|\mathbf{r}_k|$ VNFs, where $|\cdot|$ denotes set cardinality;

each VNF is run at monitoring point $i \in \mathcal{M} - \{0\}$. Define α_{ki} and β_{kj} to denote a VNF in request $\mathbf{r}_k$ that runs at monitoring point i and j , respectively. The VNFs in request $\mathbf{r}_k$ form a graph $\mathcal{G}_k(\mathbf{r}_k, \mathcal{Z}_k)$, in which $\mathcal{Z}_k$ represents the set of virtual links between VNFs and $(\alpha_{ki}, \beta_{kj}) \in \mathcal{Z}_k$ is the virtual link from VNF α_{ki} to β_{kj} in request $\mathbf{r}_k$.

Each VNF, say α_{ki} , has a tuple $(i, q_{\alpha_{ki}}, \Psi_{\alpha_{ki}}, C(\alpha_{ki}, \beta_{kj}))$, $i \in \{1, 2, \dots, M\}$, which specifies, respectively, its monitoring point, CPU frequency (in Hz), computation workload [in mega clock cycles (MCC)], and communication demand (in bit). Denote I_k as the set of VNF locations in request $\mathbf{r}_k$.

Let $b_u^t(\alpha_{ki})$ and $b_v^t(\beta_{kj})$ be binary variables to indicate whether VNF α_{ki} and β_{kj} are serviced by UAV u and v in time slot t , respectively. Define $f_{(u,v)}^{(\alpha_{ki}, \beta_{kj}), t}$ as the flow rate from VNF α_{ki} to β_{kj} when virtual link $(\alpha_{ki}, \beta_{kj}) \in \mathcal{Z}_k$ is mapped onto communication link $(u, v) \in \mathcal{K}^t$ in time slot t . The following constraint ensures that if both VNF α_{ki} and β_{kj} are embedded on UAV u and v , respectively, in time slot t , then the flow rate from VNF α_{ki} to β_{kj} is equal to the communication demand of virtual link $(\alpha_{ki}, \beta_{kj})$. Formally

$$f_{(u,v)}^{(\alpha_{ki}, \beta_{kj}), t} = b_u^t(\alpha_{ki}) b_v^t(\beta_{kj}) C(\alpha_{ki}, \beta_{kj}) \quad \forall t \in \mathcal{T}. \quad (2)$$

Define $\omega_k^t \in \{0, 1\}$ to indicate whether request $\mathbf{r}_k$ is serviced in time slot t . A request is serviced if all its VNFs are executed by a UAV in a given time slot. Mathematically

$$\sum_{u \in \mathcal{U}} \sum_{i \in \mathcal{M}} b_u^t(\alpha_{ki}) = |\mathbf{r}_k| \omega_k^t \quad \forall k \in \{1, 2, \dots, S\} \quad \forall t \in \mathcal{T}. \quad (3)$$

Each request can be serviced once. This is formalized as follows:

$$\sum_{t \in \mathcal{T}} \omega_k^t \leq 1 \quad \forall k \in \{1, 2, \dots, S\}. \quad (4)$$

In addition, each VNF α_{ki} can only be mapped onto one UAV in time slot t . Mathematically

$$\sum_{u \in \mathcal{U}} b_u^t(\alpha_{ki}) \leq 1 \quad \forall \alpha_{ki} \in \mathbf{r}_k \quad \forall \mathbf{r}_k \in \mathcal{R} \quad \forall t \in \mathcal{T}. \quad (5)$$

A UAV is active if it is running at least one VNF. Define $\theta_u^t \in \{0, 1\}$ to indicate whether UAV u is active in time slot t . Mathematically, for each α_{ki} in $\mathbf{r}_k$, we have

$$b_u^t(\alpha_{ki}) \leq \theta_u^t \leq \sum_{\alpha_{ki}} b_u^t(\alpha_{ki}) \quad \forall u \in \mathcal{U} \quad \forall t \in \mathcal{T}. \quad (6)$$

Note, if UAV u in time slot t does not run any VNF, both left- and right-hand side of (6) is zero, and consequently, we have $\theta_u^t = 0$. If UAV u in time slot t runs at least one VNF, the left-hand side of at least one of the constraints in (6) is one, and consequently $\theta_u^t = 1$.

B. Communication

All communication links are on a distinct frequency. The value of the channel power gain between two UAVs when they are hovering at monitoring points i and j , defined as $g_{(i,j)}$, is determined by the following path loss (in dB) formula [19]:

$$PL_{(i,j)} = 10\sigma \log_{10} d_{(i,j)} \quad \forall i, j \in \mathcal{M} - \{0\} \quad (7)$$

where σ is the path-loss exponent.

The transmit power of UAVs is P . The capacity of each communication link is as per the Shannon–Hartley formula

$$C_{(i,j)} = B \log_2 \left(1 + \frac{Pg(i,j)}{N_0} \right) \quad \forall i, j \in \mathcal{M} - \{0\} \quad (8)$$

where B is the channel bandwidth (in Hz) and N_0 is the noise power (in W).

In each time slot, the total flow rate on each communication link cannot exceed its link capacity, $C_{(i,j)}$, which depends on the distance between locations i and j . Thus, the communication link between UAVs u and v is determined by their location. Assume they are placed at locations i and j , respectively, at time slot t . For each VNF α_{ki} and β_{kj} in $\mathbf{r}_k$, UAV u and v in $\mathcal{U}$, time slot t in $\mathcal{T}$, and locations i and j in $\mathcal{M} - \{0\}$, we have

$$\sum_{k=1}^S \sum_{\alpha_{ki} \in \mathbf{r}_k} \sum_{\beta_{kj} \in \mathbf{r}_k} \left(f_{(u,v)}^{(\alpha_{ki}, \beta_{kj}), t} + f_{(v,u)}^{(\beta_{kj}, \alpha_{ki}), t} \right) \leq C_{(i,j)}. \quad (9)$$

Denote $\rho_{(i,j)}$ as the average power to transmit one bit from monitoring point i to j . Formally

$$\rho_{(i,j)} = \eta + \frac{10^{\frac{\epsilon-3}{10}} g_{(i,j)}}{\gamma} \quad \forall i, j \in \mathcal{M} - \{0\} \quad (10)$$

where η indicates transceiver circuit power (in W), ϵ is the receiver sensitivity of the power amplifier (in dBm), and γ is the drain efficiency of the power amplifier [20]. The channel power gain between UAV u and v at time slot t is positive only if both UAV u and v are located at locations i and j .

C. UAV Movement

Define $\mu_u^t(i) \in \{0, 1\}$ to indicate whether UAV u is stationed at location i in time slot t . For each UAV u in $\mathcal{U}$ and time slot t in $\mathcal{T}$, we have

$$b_u^t(\alpha_{ki}) \leq \mu_u^t(i) \quad \forall \alpha_{ki} \in \mathbf{r}_k \quad \forall \mathbf{r}_k \in \mathcal{R} \quad \forall i \in \mathcal{M} - \{0\}. \quad (11)$$

In each time slot t , each UAV u is located at one location

$$\sum_{i \in \mathcal{M}} \mu_u^t(i) = 1 \quad \forall u \in \mathcal{U} \quad \forall t \in \mathcal{T}. \quad (12)$$

Further, each monitoring point has one UAV

$$\sum_{u \in \mathcal{U}} \mu_u^t(i) \leq 1 \quad \forall i \in \mathcal{M} - \{0\} \quad \forall t \in \mathcal{T}. \quad (13)$$

The next set of constraints models whether a UAV travels from its current location to another location in $\mathcal{M}$. Define $\pi_u^{t,t+1}(i, j) \in \{0, 1\}$ to indicate whether UAV u travels over path $(i, j) \in \mathcal{D}$ between time slot t and $t+1$. For each UAV u in $\mathcal{U}$, j in $\mathcal{M}$, and time slot $t \in \{1, 2, \dots, T-1\}$, we have

$$\mu_u^t(i) + \mu_u^{t+1}(j) - 1 \leq \pi_u^{t,t+1}(i, j) \quad (14)$$

$$\pi_u^{t,t+1}(i, j) \leq \mu_u^t(i) \quad (15)$$

$$\pi_u^{t,t+1}(i, j) \leq \mu_u^{t+1}(j). \quad (16)$$

As an example, assume that UAV u moves from location i to j . So we will have $\mu_u^t(i) = \mu_u^{t+1}(j) = 1$. Consequently, the above constraints result in $\pi_u^{t,t+1}(i, j) = 1$. On the other hand, if UAV u located on location i does not travel to location j ,

we have $\pi_u^{t,t+1}(i, j) = 0$. This means UAV u does not incur any cost relating to communication link (u, v) .

Note, if a UAV is not running VNF(s) and it is located above a monitoring point in $\mathcal{M}$, then it lands at its deployed monitoring point to conserve energy.

D. Energy Consumption

UAVs consume energy when they are 1) traveling and 2) servicing VNF(s), which includes the energy cost of hovering and running VNF(s).

1) *Traveling*: UAVs have a fixed traveling time, $\bar{T}$, which can be calculated based on the maximum distance between locations and the maximum speed of UAVs. The speed to travel from location i to j , i.e., velocity $V_{(i,j)}$, is

$$V_{(i,j)} = \frac{d_{(i,j)}}{\bar{T}} \quad \forall i, j \in \mathcal{M}. \quad (17)$$

Consequently, the total required energy consumption per unit traveling distance (in J/m), $\Gamma_{(i,j)}$, as per [21] is

$$\begin{aligned} \Gamma_{(i,j)} = & \kappa \left(\frac{1}{V_{(i,j)}} + \frac{3V_{(i,j)}}{\lambda^2} \right) \\ & + \iota \left(\sqrt{V_{(i,j)}^{-4} + \frac{1}{4v_0^4}} - \frac{1}{2v_0^2} \right)^{1/2} \\ & + \frac{1}{2} \nu \Delta s \Omega V_{(i,j)}^2 \quad \forall i, j \in \mathcal{M} \end{aligned} \quad (18)$$

where λ is the tip speed of the rotor blade, v_0 is the mean rotor induced velocity in hover, ν is the fuselage drag ratio, Δ is air density, s is the rotor solidity, and Ω is the rotor disc area. Note, the term $d_{(i,j)} \Gamma_{(i,j)}$ indicates traveling cost from location i to j between time slots.

2) *Servicing VNFs*: At a deployed location, a UAV consumes energy for hovering, running VNF(s), receiving and transmitting data. These costs are defined as follows.

- 1) Let $\tau(\kappa + \iota)\theta_u^t$ be the total energy UAV u uses to hover in time slot t , where κ and ι indicate the blade profile power and induced power during hovering [21].
- 2) Define the term $h \sum_{k=1}^S \sum_{\alpha_{ki} \in \mathbf{r}_k} q_{\alpha_{ki}}^2 \Psi_{\alpha_{ki}} b_u^t(\alpha_{ki})$ as the total energy that UAV u consumes to process VNF(s) in time slot t , where h is a constant which depends on the chip architecture of the UAV; i.e., $h = 10^{-11}$ [22], [23].
- 3) The total energy that each UAV consumes to receive data in each time slot is $\tau\eta\theta_u^t$.
- 4) Define the total energy that UAV u consumes for transmitting data in time slot t as $\tau \sum_{v \in \mathcal{N}_u^t} \sum_{k=1}^S \sum_{(\alpha_{ki}, \beta_{kj}) \in \mathcal{Z}_k} \rho_{(i,j)} f_{(u,v)}^{(\alpha_{ki}, \beta_{kj}), t}$.

Let ξ_u^t denote the UAV energy consumption if it is active in time slot t . For all UAV u in $\mathcal{U}$ and for all time slot t in $\mathcal{T}$, we have

$$\begin{aligned} \xi_u^t = & \tau(\kappa + \iota)\theta_u^t + h \sum_{k=1}^S \sum_{\alpha_{ki} \in \mathbf{r}_k} q_{\alpha_{ki}}^2 \Psi_{\alpha_{ki}} b_u^t(\alpha_{ki}) + \tau\eta\theta_u^t \\ & + \tau \sum_{v \in \mathcal{N}_u^t} \sum_{k=1}^S \sum_{(\alpha_{ki}, \beta_{kj}) \in \mathcal{Z}_k} \rho_{(i,j)} f_{(u,v)}^{(\alpha_{ki}, \beta_{kj}), t}. \end{aligned} \quad (19)$$

Note, if UAV u is not active in time slot t , we have $\xi_u^t = 0$.

E. Battery Evolution

If a UAV travels to the base point, its battery is replaced with a fully charged battery. Let E_u^t denote the energy level of UAV u at the beginning of time slot t . Define $E_{\max}$ as the maximum battery capacity of each UAV. We have

$$0 \leq E_u^t \leq E_{\max} \quad \forall u \in \mathcal{U} \quad \forall t \in \mathcal{T}. \quad (20)$$

In each time slot, the energy level at UAVs is a function of their previous energy level, whether they were at the base point, hovering, traveling, processing VNF(s), and/or communicating with other UAVs. More specifically, at the beginning of the time horizon, all UAVs are located at the base point with the maximum energy, $E_{\max}$. They may travel to monitoring points to service VNFs before the start of the first time slot. At the end of the last time slot, all UAVs located at monitoring points travel to the base point. For each UAV u in $\mathcal{U}$ at the beginning of the first time slot, we have

$$E_u^1 = E_{\max} - \sum_{(0,i) \in \mathcal{D}} (\mu_u^1(i) d_{(0,i)} \Gamma_{(0,i)}) \quad \forall i \in \mathcal{M} - \{0\}. \quad (21)$$

For time slot t , the energy level of UAV u evolves as per

$$E_u^t = E_u^{t-1} + a_u^{t-1} - \xi_u^{t-1} - \sum_{(i,j) \in \mathcal{D}} \pi_u^{t-1,t}(i,j) d_{(i,j)} \Gamma_{(i,j)} \quad (22)$$

where a_u^t indicates the difference between the capacity of new battery and the energy level of the previous battery in case of replacing battery at the base point in time slot t .

To prevent battery overflow, we have the following constraints for each UAV u in $\mathcal{U}$ and each time slot t in $\mathcal{T}$:

$$E_{\max} - E_u^t - (1 - \mu_u^t(0))M \leq a_u^t \leq E_{\max} - E_u^t \quad (23)$$

$$0 \leq a_u^t \leq \mu_u^t(0)M \quad (24)$$

where M is a constant used to disable a constraint; it can be set to $E_{\max}$. If UAV u is at the base point, $\mu_u^t(0)$ is one, then we have $a_u^t = E_{\max} - E_u^t$, which ensures that there is no overflow. Otherwise, it is $a_u^t = 0$.

In each time slot, the energy consumption of each UAV cannot exceed its energy level. Thus, for each UAV u in $\mathcal{U}$ and each time slot t in $\{1, 2, \dots, T-1\}$, we have

$$-a_u^t + \xi_u^t + \sum_{(i,j) \in \mathcal{D}} (\pi_u^{t,t+1}(i,j) d_{(i,j)} \Gamma_{(i,j)}) \leq E_u^t \quad (25)$$

For each UAV u in $\mathcal{U}$ in time slot T , we have

$$-a_u^T + \xi_u^T + \sum_{(0,i) \in \mathcal{D}} (\mu_u^T(i) d_{(0,i)} \Gamma_{(0,i)}) \leq E_u^T. \quad (26)$$

F. Computation

In time slot t , the computation workload of UAV u cannot exceed its computation capacity, $\Psi_{\max}$. Formally

$$\sum_{k=1}^S \sum_{\alpha_{ki} \in \mathbf{r}_k} \Psi_{\alpha_{ki}} b_u^t(\alpha_{ki}) \leq \Psi_{\max} \quad \forall u \in \mathcal{U} \quad \forall t \in \mathcal{T}. \quad (27)$$

IV. PROBLEM STATEMENT

The aim is to maximize the reward, e.g., revenue, of an operator. This is achieved by satisfying as many requests as possible. To do that, a key problem is to determine which UAVs to deploy to service which requests at which time slot. Mathematically, this problem can be modeled as follows:

$$\begin{aligned} & \text{maximize} \quad \sum_{t=1}^T \sum_{k=1}^S W_k \omega_k^t \\ & \text{subject to} \quad (3)-(6), (9), (11)-(16) \\ & \quad \quad \quad (20), (23)-(27). \end{aligned} \quad (28)$$

Solving MILP (28) is challenging as its solution space grows exponentially with the number of monitoring points, the number of requests, the number of VNFs per request, and planning horizon. In fact, a special case of the problem considered in this article is a variant of the well-known traveling salesman problem (see Section VI).

Next, we outline a number of solutions for large problem instances, where we outline heuristic methods for deploying requests within each time slot and assigning one or more UAVs to service requests.

V. HEURISTIC SOLUTIONS

We propose a heuristic solution called MPopLoc. The objective is to reduce traveling cost by placing UAVs at locations with more VNFs in order to save energy and decrease the frequency of recharging. MPopLoc contains two phases: 1) *Resource Commitment* and 2) *Resource Allocation*. First, MPopLoc selects the most popular request based on a popularity metric. In the *Resource Commitment* phase, MPopLoc selects UAVs with the minimum traveling cost and checks if there are sufficient resources, including communication, energy, and computation, to service a selected request. If there are sufficient resources, in the *Resource Allocation* phase, MPopLoc allocates selected UAVs to locations specified in the selected request. It then updates the energy level and computation capacity of allocated UAVs and communication capacity of links.

Next, using Algorithm 1, we explain the details of MPopLoc. Denote $\mathcal{R}^t$ as the set of serviced requests in time slot t . We use $\mathcal{U}'$ to indicate potential UAVs for servicing a selected request and $\mathcal{U}_{ch}^t$ to represent charging UAVs at the base point in time slot t . Note that if the energy level and location of UAVs change, MPopLoc updates E^t and $\mathcal{L}^t$ in lines 21 and 22, respectively. First, MPopLoc updates the set $\mathcal{R}$ to include only those requests awaiting service (see line 8). In terms of the popularity of each location, we define a counter for each location as $\text{ploc}(i)$, indicating how many requests require location i . Next, we establish a counter to measure the popularity of each request. We have

$$\text{preq}(\mathbf{r}_k) = \sum_{i \in I_k} \text{ploc}(i) \quad \forall \mathbf{r}_k \in \mathcal{R}. \quad (29)$$

Line 9 sorts unserved requests based on their popularity in descending order.

Algorithm 1 MPopLoc**Input:** $\mathcal{T}, \mathcal{U}, E_{\max}, \Psi_{\max}, \mathcal{R}$ **Output:** $\mathcal{L}, \mathcal{R}$

```

1:  $\mathcal{R}^0 = \mathcal{U}_{ch}^1 = \emptyset$ 
2:  $\mathcal{L}^0 = 0$ 
3:  $E^0 = E_{\max}$ 
4: for each  $t \in \mathcal{T}$  do
5:    $\mathcal{U}^t = \mathcal{R}^t = \emptyset$ 
6:    $E^t = E^{t-1}$ 
7:    $\mathcal{L}^t = \mathcal{L}^{t-1}$ 
8:    $\mathcal{R} = \mathcal{R} - \mathcal{R}^{t-1}$ 
9:    $\mathcal{R} = \text{sort}(\mathcal{R})$ 
10:  for each  $\mathbf{r}_k \in \mathcal{R}$  do
11:    *** Resource Commitment Phase ***
12:     $[SFlag, \mathcal{U}_{ch}^{t+1}, \mathcal{U}'] = \text{ResCommit}(z)$ 
13:    if  $SFlag == \text{True}$  then
14:      *** Resource Allocation Phase ***
15:       $\mathcal{U}' \leftarrow \mathcal{U}'$ 
16:       $\mathcal{R}^t \leftarrow \mathbf{r}_k$ 
17:      for  $i \in I_k$  and  $u \in \mathcal{U}'$  do
18:        for  $j \in I_k$  do
19:           $C_{(i,j)} = C_{(i,j)} - C_{(\alpha_{ki}, \beta_{kj})}$ 
20:        end for
21:         $E_u^t = E_u^t - d_{(\mathcal{L}_u^t, i)} \Gamma_{(\mathcal{L}_u^t, i)} + \xi_{\alpha_{ki}}$ 
22:         $\mathcal{L}_u^t = i$ 
23:         $\Psi_{\max} = \Psi_{\max} - \Psi_{\alpha_{ki}}$ 
24:      end for
25:    end if
26:  end for
27:  for  $u \in \mathcal{U}_{ch}^t$  do
28:     $\mathcal{L}_u^t = 0$ 
29:     $E_u = E_{\max}$ 
30:  end for
31:   $\mathcal{U}_{ch}^t = \emptyset$ 
32:   $\mathcal{L} = \mathcal{L} \cup \mathcal{L}^t$ 
33:   $\mathcal{R} = \mathcal{R} \cup \mathcal{R}^t$ 
34: end for
35: Return  $\mathcal{L}, \mathcal{R}$ .

```

In the *Resource Commitment* phase, in line 12, MPopLoc calls *ResCommit()*, see Section V-A, to check requests from the most popular to the least popular location to determine if they can be serviced. Note that $z = [\mathbf{r}_k, \Psi_{\max}, \mathcal{U}^t, \mathcal{U}_{ch}^t, \mathcal{L}^t, E^t]$ in line 12 represents the inputs of *ResCommit()*.

If a request can be serviced, MPopLoc proceeds to the *Resource Allocation* phase (see lines 13–25). In this phase, MPopLoc saves the list of potential UAVs as deployed UAVs (line 15) and request $\mathbf{r}_k$ as a serviced request (see line 16). It also updates the amount of traffic routed over each communication link, energy level, location, and computation of allocated UAVs (lines 17–24). Note that in line 21, MPopLoc calculates the energy consumption of allocated UAVs to service VNF α_{ki} using the following formula:

$$\xi_{\alpha_{ki}} = F_i^t \tau (\kappa + \iota) + h q_{\alpha_{ki}}^2 \Psi_{\alpha_{ki}} + \tau \eta + \tau \sum_{\beta_{kj} \in \mathbf{r}_k} \rho_{(i,j)} C_{(\alpha_{ki}, \beta_{kj})} \quad (30)$$

where F_i^t is a binary variable to indicate whether a UAV is initially deployed at location i during time slot t .

If request $\mathbf{r}_k$ cannot be serviced, MPopLoc discards the request, selects the next unserved request in $\mathcal{R}$, and repeats the previous steps.

After checking all unserved requests, MPopLoc updates the location and energy level of charging UAVs in lines 27–30. Then, it proceeds to the next time slot $t + 1$, i.e., repeating line 4 in Algorithm 1. Finally, in line 35, MPopLoc returns μ and $\mathcal{R}$, which indicate UAV placement and the set of serviced requests in each time slot, as assigned in lines 32 and 33.

A. ResCommit()

Algorithm 2 shows the steps of *ResCommit()*. It calculates the communication capacity between the required locations using (8) and checks if there is sufficient communication capacity to service request $\mathbf{r}_k$ (see lines 2–9). If so, Algorithm 2 proceeds to select UAVs for the required locations to service request $\mathbf{r}_k$ based on their traveling cost $d_{(\mathcal{L}_u^t, i)} \Gamma_{(\mathcal{L}_u^t, i)}$, which is the amount of energy that UAV u consumes to travel from its current location $\mathcal{L}_u^t$ to the required location i . The function *ResCommit()* has two phases to select UAVs.

- 1) *Phase 1*: To reduce traveling cost, priority is given to already deployed UAVs at required locations. Therefore, in this phase, *ResCommit()* checks if there is a UAV at each location in I_k (see line 14). Note that *ResCommit()* calculates the energy consumption of UAVs using the following formula:

$$E_u^t = d_{(\mathcal{L}_u^t, i)} \Gamma_{(\mathcal{L}_u^t, i)} + \xi_{\alpha_{ki}} + d_{(i,0)} \Gamma_{(i,0)}. \quad (31)$$

For locations with a UAV, if the UAV assigned to a location does not have sufficient energy, it travels to the base point for recharging in the next time slot (see lines 15 and 16). If the UAV assigned to a location has sufficient energy and computation capacity, it is saved in set $\mathcal{U}'$ (see lines 18 and 19).

- 2) *Phase 2*: In this phase, *ResCommit()* selects UAVs for locations without a UAV. To do so, *ResCommit()* first updates the list of UAVs to have UAVs that can be assigned (see line 28). Then, it sorts them based on their traveling cost from their current location to the target location in descending order; line 29. In lines 30–39, if a UAV does not have sufficient energy, it travels to the base point for recharging in the next time slot (see lines 32 and 33). If a UAV has sufficient energy, it is saved in set $\mathcal{U}'$ (see line 35).

If the number of locations in the selected request is not equal to the number of UAVs saved in set $\mathcal{U}'$ (line 42), it means that not all locations in the selected request have a corresponding UAV, indicating insufficient resources to service request $\mathbf{r}_k$. Conversely, if the numbers are equal, it confirms the availability of sufficient resources to service request $\mathbf{r}_k$. Finally, *ResCommit()* returns *SFlag*, $\mathcal{U}_{ch}^{t+1}$, and $\mathcal{U}'$ in line 45.

VI. ANALYSIS

This section outlines propositions relating to the run-time complexity of the proposed heuristic solutions. After that, it

Algorithm 2 *ResCommit()*

Input: $\mathbf{r}_k, \Psi_{\max}, \mathcal{U}^t, \mathcal{U}_{ch}^t, \mathcal{L}^t, E^t$
Output: $SFlag, \xi_{\alpha_{ki}}, \mathcal{U}_{ch}^{t+1}$

```

1:  $\mathcal{U}' = \emptyset$ 
2:  $SFlag = \text{False}$ 
3: for  $i \in I_k$  do
4:   for  $j \in I_k$  do
5:     if  $C_{(\alpha_{ki}, \beta_{kj})} \leq C_{(i,j)}$  then
6:        $SFlag = \text{True}$ 
7:     end if
8:   end for
9: end for
10: if  $SFlag == \text{True}$  then
11:   *** Phase 1 ***
12:   for each  $i \in I_k$  do
13:     for each  $u \in \mathcal{U}$  do
14:       if  $\mathcal{L}_u^t == i$  then
15:         if  $E_u^t < E_u(\alpha_{ki})$  then
16:            $\mathcal{U}_{ch}^{t+1} \leftarrow u$ 
17:         else
18:           if  $\Psi_{\max} \geq \Psi_{\alpha_{ki}}$  then
19:              $\mathcal{U}' \leftarrow u$ 
20:           end if
21:         end if
22:       end if
23:     end for
24:   end for
25:   *** Phase 2 ***
26:   for each  $i \in I_k$  do
27:      $\mathcal{U}'' = \mathcal{U}$ 
28:      $\mathcal{U}'' = \mathcal{U}'' - \mathcal{U}' - \mathcal{U}^t - \mathcal{U}_{ch}^t$ 
29:      $\mathcal{U}'' = \text{sort}(\mathcal{U}'')$ 
30:     for each  $u \in \mathcal{U}''$  do
31:       if  $\mathcal{L}_u^t \neq i$  then
32:         if  $E_u^t < E_u(\alpha_{ki})$  then
33:            $\mathcal{U}_{ch}^{t+1} \leftarrow u$ 
34:         else
35:            $\mathcal{U}' \leftarrow u$ 
36:           Break
37:         end if
38:       end if
39:     end for
40:   end for
41: end if
42: if  $|I_k| \neq |\mathcal{U}'|$  then
43:    $SFlag = \text{False}$ 
44: end if
45: Return  $SFlag, \mathcal{U}_{ch}^{t+1}, \mathcal{U}'$ .

```

presents an analysis of the hardness of the problem outlined in Section IV.

Proposition 1: Algorithm 2 has a run time of $\mathcal{O}(|U|^2)$.

Proof: *ResCommit()* runs the loop from lines 3–9 for $|I_k|^2$ times. In Phase 1, the loop from lines 12–24 is repeated $|I_k||U|$ times. In Phase 2, first line in the loop from lines 26–40 iterate

through each element in I_k . Inside the loop, line 29, which is for sorting operation for $|U''|$ items, has a time complexity of $\mathcal{O}(|U''| \log(|U''|))$ and the loop from lines 30–39 iterates through each element in U'' times. Therefore, the time complexity of Phase 2 is $\mathcal{O}(|I_k|(|U''| \log(|U''|) + |U''|))$. Overall, the time complexity of Algorithm 2 is $\mathcal{O}(|I_k|^2 + |I_k||U| + |I_k|(|U''| \log(|U''|) + |U''|))$. Note that $|I_k|$ is the number of VNF locations in request r_k , which is assumed to be no more than the number of UAVs, $|U|$. Therefore, the number of VNF locations that a request can have is $|U|$ at most. On the other hand, the number of UAVs which can be assigned to a location, $|U''|$, is $|U|$ at most. Additionally, for $|U| \geq 1$, we have $\mathcal{O}(|U|^2 + |U| \log(|U|)) = \mathcal{O}(|U|^2)$. Consequently, the overall complexity is $\mathcal{O}(|I_k|^2 + |I_k||U| + |I_k|(|U''| \log(|U''|) + |U''|))$ or simply $\mathcal{O}(|U|^2)$. ■

Proposition 2: MPopLoc has a run time of $\mathcal{O}(|U|^3)$.

Proof: Algorithm 1 runs the loop from lines 4–34 for $|\mathcal{T}|$ times. Sorting the set $\mathcal{R}$ in line 9 has a run time complexity of $\mathcal{O}(|\mathcal{R}| \log(|\mathcal{R}|))$. Line 10 iterates through each element in $\mathcal{R}$. The *Resource Commitment* phase calls *ResCommit()*, which has complexity $\mathcal{O}(|U|^2)$. In the *Resource Allocation* phase, the loop from lines 17–24 is executed $|I_k|^2|U'|$ times. Therefore, the loop from lines 10–26 iterates $|\mathcal{R}|(|U|^2 + |I_k|^2|U'|)$ times. Lines 27–30 contain a loop that is run $|U_{ch}^t|$ times. Overall, the time complexity of Algorithm 1 is $\mathcal{O}(|\mathcal{T}|(|\mathcal{R}| \log(|\mathcal{R}|) + |\mathcal{R}| + (|\mathcal{R}|(|U|^2 + |I_k|^2|U'|) + |U_{ch}^t|)))$. Note that $|U'|$ and $|U_{ch}^t|$, which represent the number of potential UAVs available for servicing a selected request and the number of charging UAVs at the base point in time slot t , respectively, are $|U|$ at most. As a result, the overall complexity is $\mathcal{O}(|\mathcal{T}|(|\mathcal{R}| \log(|\mathcal{R}|) + |\mathcal{R}| + (|\mathcal{R}|(|U|^2 + |I_k|^2|U'|) + |U_{ch}^t|)))$ or simply $\mathcal{O}(|U|^3)$. ■

Finally, consider a special case of the following NP-hard CCTSP problem [24]: given a set of cities, each with a value, and traveling cost between cities. The goal is to find a subtour of these cities that yields the maximum value subject to a given budget on traveling cost. Consider now the following problem. There are S requests that require a VNF to be deployed in a distinct location or “cities.” This means there are S locations or cities. Each request has a payment or value. Finally, assume there is one UAV which has a given energy budget. This is exactly the same as CCTSP, which implies that the problem addressed in this article is at least as hard as CCTSP.

VII. EVALUATION

The evaluation methodology to study the aforementioned problem, formulation and heuristic solutions is conducted using Python 3.10.9 and Gurobi 10.0.1 on an Intel i7-8700 CPU @ 3.20-GHz computer. Table III summarizes parameter values; these parameters are defined in Section III and they serve as input data or constants. Unless otherwise stated, the evaluation uses following parameter values. There are ten locations on a $100 \times 100 \text{ m}^2$ area. The planning horizon has four time slots; each time slot is 1 s in length. There are four UAVs with a maximum speed of $\bar{V} = 15 \text{ m/s}$ and battery capacity $E_{\max} = 277 \text{ 200 J}$; see DJI Mavic 3 specification [25].

TABLE III
SUMMARY OF PARAMETERS VALUES

Parameter	Value
Time slot duration (τ)	1 s
Traveling time ($\bar{T}$)	9.43 s
CPU frequency of each VNF ($q_{\alpha_{ki}}$)	[0.1, 0.5] GHz
Computation capacity of each UAV (Ψ_{max}) [28]	1 GHz
Computation workload ($\Psi_{\alpha_{ki}}$)	[100, 500] MCCs
Communication demand ($C_{(\alpha_{ki}, \beta_{kj})}$)	[1, 5] $\times 10^5$ bits
Blade profile power during hover (κ) [21]	79.86 W
Induced power during hover (ι) [21]	88.63 W
Max UAV traveling speed ($\bar{V}$) [25]	15 m/s
Tip speed of rotor blade (λ) [21]	120 m/s
Mean rotor induced velocity in hover (v_0) [21]	4.03 m/s
Fuselage drag ratio (ν) [21]	0.6
Air density (Δ) [21]	1.225 kg/m ³
Rotor solidity (s) [21]	0.05
Rotor disc area (Ω) [21]	0.503 m ²
Circuit power (η) [27]	59.8 mW
Receiver sensitivity (ϵ) [27]	-85 dBm
Drain efficiency of a power amplifier (γ) [27]	0.05
Maximum battery capacity (E_{max}) [25]	277200 J
Path-loss exponent (σ) [26]	2.05
Channel bandwidth (B)	200 kHz
UAV transmit power (P)	1 W
Noise power (N_0)	-95 dBm

The traveling time $\bar{T}$ is based on a UAV's maximum travel distance and velocity; formally, $\bar{T} = \sqrt{(100^2 + 100^2)}/15$. During hovering, the UAV's blade profile power and induced power are set as per [21], i.e., $\kappa = 79.86$ W and $\iota = 88.63$ W. In terms of traveling cost, the evaluation uses m/s, $\lambda = 120$ m/s, $v_0 = 4.03$ m/s, $\nu = 0.6$, $\Delta = 1.225$ kg/m³, $s = 0.05$, and $\Omega = 0.503$ m² [21]. UAVs have a 1 GHz processor and transmit power of $P = 30$ dBm.

There are 20 requests, each has two VNFs that should be run on two different locations. All requests have the same revenue, which is set to \$1000. The CPU frequency of each VNF $q_{\alpha_{ki}}$ is drawn randomly from the range [0.1, 0.5] (in GHz). The computation workload of each VNF $\Psi_{\alpha_{ki}}$ is chosen randomly from [100, 500] (in MCCs). The communication demand between VNFs is drawn randomly from the range [1 $\times 10^5$, 5 $\times 10^5$] (in bits). As per [26], σ is set to 2.05. For communication related parameters, the evaluation uses $\eta = 59.8$ mW, $\epsilon = -85$ dBm, and $\gamma = 0.05$ [27]. The noise power is set to $N_0 = -95$ dBm.

The following factors are studied: 1) the number of UAVs; 2) computation capacity; 3) computation workload; 4) battery capacity; 5) the number of time slots; 6) communication demand; 7) the number of VNFs in each request; 8) the number of requests; and 9) revenue. The evaluation also considers another heuristic algorithm named LPopLoc. The distinction between MPopLoc and LPopLoc lies in request selection. In MPopLoc, the goal is to prioritize requests from more popular locations, while LPopLoc prioritizes requests from less popular locations. Additionally, MILP, MPopLoc, and LPopLoc are compared against a benchmark algorithm named RAND. This algorithm allocates available UAVs to monitoring points randomly in each time slot. It then checks all unserved requests in a random order, attempting to service as many requests as possible. The results are an average of 50 runs.

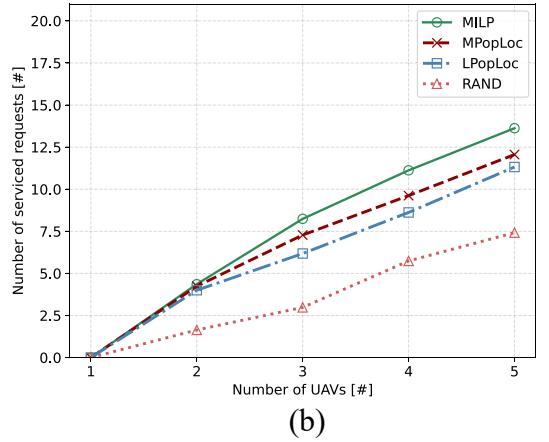
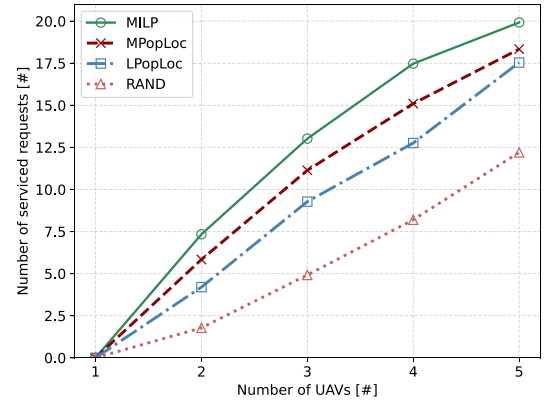


Fig. 2. Number of requests serviced by MILP, MPopLoc, LPopLoc, and RAND with different numbers of UAVs and computation capacity. (a) Computation capacity: 1 GHz. (b) Computation capacity: 500 MHz.

A. Computation Capacity

The evaluation first studies how varying computation capacity Ψ_{max} and the number of UAVs U impact the number of serviced requests. In both Fig. 2(a) and (b), the number of serviced requests increases with the number of UAVs, given that there are more computation and energy resources. Further, it is more likely to have a UAV at a location. For example, when UAVs have a computation capacity of 1 GHz, MILP is able to service all 20 requests. In contrast, MPopLoc and LPopLoc are able to service 18.34 and 17.54 requests on average, respectively. At 500 MHz, MILP, MPopLoc, and LPopLoc managed only 13.8, 12.06 and 11.32 requests on average, respectively.

B. Computation Workload

Computation workload $\Psi_{\alpha_{ki}}$ and the number of UAVs U have an impact on the number of serviced requests. Referring to Fig. 3(a), when the computation workload is 500 MCC, there is an increase in the number of serviced requests. This is because with more UAVs, there will be more resources and location coverage. In Fig. 3(b), when the computation workload is doubled to 1000 MCCs, there is an increase in the number of serviced requests. However, this increase is comparatively smaller than the one observed in Fig. 3(a). The

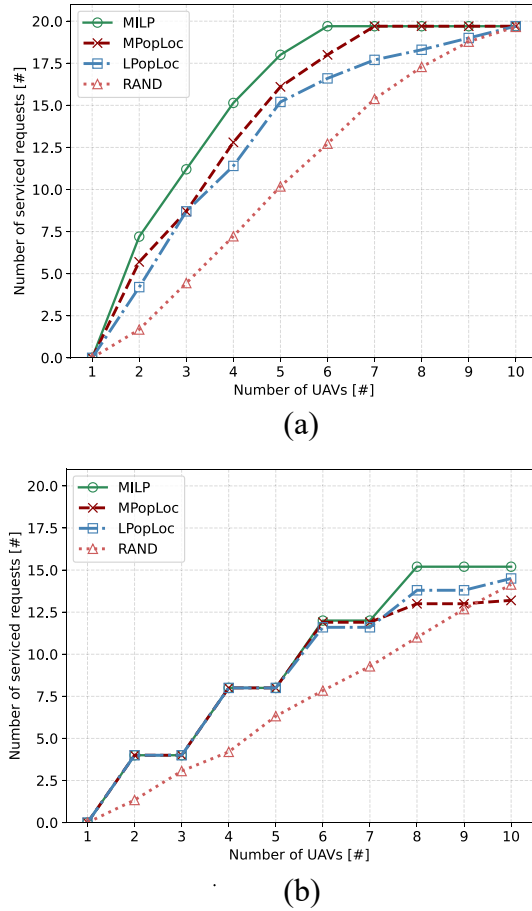


Fig. 3. Number of requests serviced by MILP, MPopLoc, LPopLoc, and RAND with different number of UAVs and computation workload. (a) Computation workload: 500 MCCs. (b) Computation workload: 1000 MCCs.

reason is that doubling the computation workload at this point pushes UAVs beyond their computational and energy limits. RAND performs poorly because UAVs may be placed at a location with no VNF or not part of any requests. In Fig. 3(b), MPopLoc outperforms LPopLoc, except when using eight, nine, and ten UAVs. The reason is that after servicing only one VNF, it may end up blocking other requests to be serviced in the current time slot. This is because each UAV cannot service more than one VNF due to its limited computation resource, making it unavailable for other VNFs. As MPopLoc allocates UAVs to locations with more VNFs, it blocks more requests. In contrast, as LPopLoc starts with less popular requests, it blocks fewer requests and consequently yields better results.

To illustrate, consider the example in Fig. 4. MPopLoc starts with r_2 , which is the most popular request. After servicing r_2 , MPopLoc cannot service more requests because the UAVs at location M_2 and M_3 have used all their computation capacity to service the first request. Consequently, they have no computation capacity to service requests r_1 and r_3 , which include VNFs at location M_2 and M_3 , respectively. However, LPopLoc can service two less popular requests r_1 and r_3 .

Finally, MILP, MPopLoc, and LPopLoc have a staircase trend. In contrast, RAND does not have a staircase trend. This

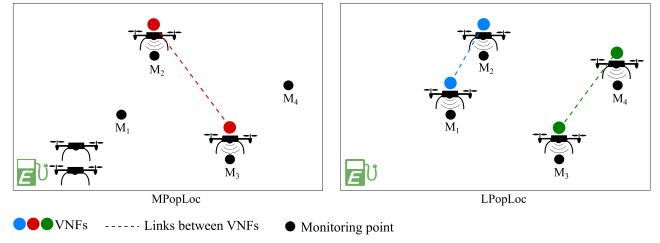


Fig. 4. Example of servicing request $r_1 = \{M_1, M_2\}$, $r_2 = \{M_2, M_3\}$, and $r_3 = \{M_3, M_4\}$ using four UAVs in one time slot, using MPopLoc and LPopLoc.

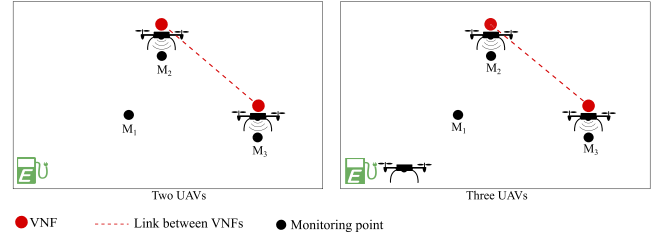


Fig. 5. Example of servicing request $r_1 = \{M_1, M_2\}$ and $r_2 = \{M_2, M_3\}$ using two and three UAVs, employing MPopLoc.

can be explained as follows. First, recall that RAND places UAVs at random locations. This means UAVs can be placed at locations without a request. However, with more UAVs, it is more likely that any two UAVs are placed at locations with a request. In contrast, the staircase trend for MILP, MPopLoc, and LPopLoc is due to the fact that the computation workload demanded, which is 1000 MCCs, fully utilizes the available computational resource on UAVs, which is limited to 1 GHz. This leaves UAVs with no additional capacity to handle other requests simultaneously. With two VNFs in each request, all requests require even number of UAVs. Otherwise, if there are odd number of UAVs, then the odd numbered UAV will not improve performance. To illustrate, consider the example in Fig. 5, which uses MPopLoc in two scenarios with two and three UAVs. Both scenarios can only service one request, namely, r_1 . Although there is a free UAV at the base point in the three-UAV scenario, it cannot help service request r_2 because UAV at location M_2 has reached its computation capacity limit.

C. Battery Capacity

The battery capacity $E_{\max}$ of UAVs has an impact on the number of serviced requests. As shown in Fig. 6, the number of serviced requests increases with higher battery capacity. This is expected because UAVs will have more energy to perform tasks, including traveling, hovering, processing VNFs, and communication. In addition, it reduces recharging frequency, which takes one time slot. Consequently, UAVs are available to process VNFs in more time slots. However, due to limitations in computation and communication capacities, as well as the number of time slots and UAVs, it is not possible to service all 20 requests, even when UAVs have a battery capacity exceeding 200 000 J.

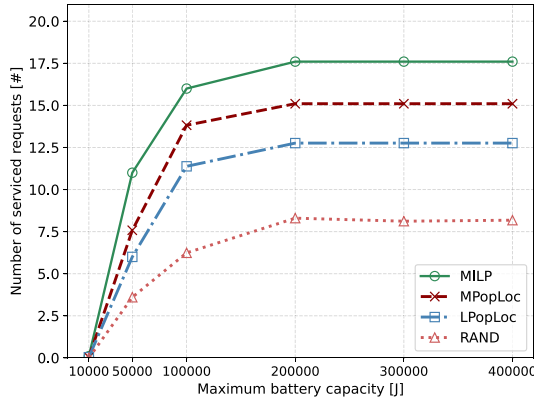


Fig. 6. Number of requests serviced by MILP, MPopLoc, LPopLoc, and RAND with different battery capacity values.

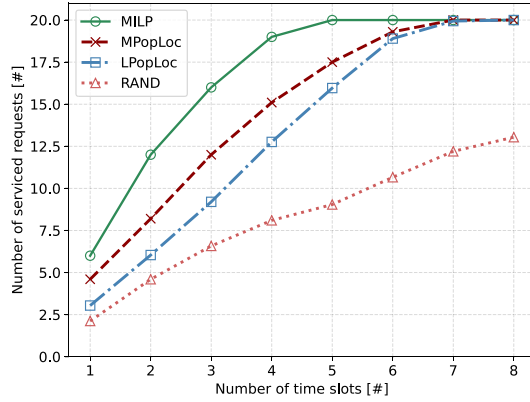


Fig. 7. Number of requests serviced by MILP, MPopLoc, LPopLoc, and RAND with different number of time slots.

D. Number of Time Slots

As shown in Fig. 7, more requests are serviced with increasing number of time slots. This is because UAVs have opportunities to recharge, thereby allowing them to serve more requests. Consequently, they have sufficient energy to service more requests. Additionally, given more time slots, they are able to service more requests. MILP needs at least five time slots to service all 20 requests. In contrast, both MPopLoc and LPopLoc require seven time slots to service all requests.

E. Communication Demand

As shown in Fig. 8, the number of serviced requests decreases with higher communication demand. This decrease is due to limited communication capacity. Another reason is higher communication demand, which results in higher amount of energy consumption by UAVs for data transmissions.

F. Number of VNFs

As shown in Fig. 9, the number of serviced requests decreases with the number of VNFs in each request. This is because requests have more VNFs, which require more resources, including computation, communication, energy, UAVs, and time slots. In addition, as all the VNFs in each request have to be serviced in one time slot, more UAVs have to be connected at the same time. Therefore, as there

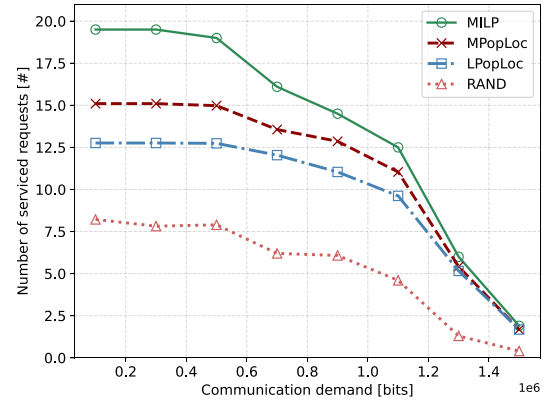


Fig. 8. Number of requests serviced by MILP, MPopLoc, LPopLoc, and RAND with different communication demands.

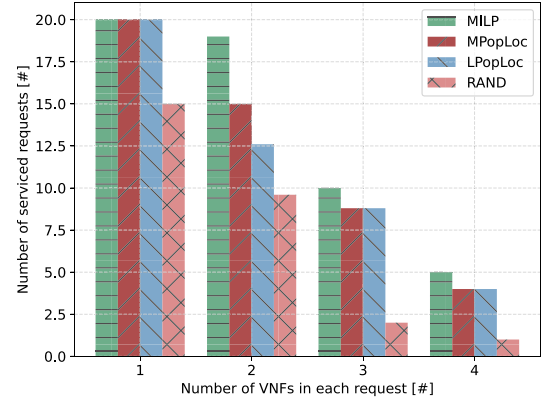


Fig. 9. Number of requests serviced by MILP, MPopLoc, LPopLoc, and RAND with different number of VNFs in each request.

are a limited number of UAVs and available resources, fewer requests will be serviced.

The gap between MPopLoc and LPopLoc narrows as the number of VNFs increases. The reason is that with more VNFs in each request, all locations are popular. Therefore, the difference between choosing more popular requests or less popular requests decreases. For example, when each of the 20 requests has four VNFs, both MPopLoc and LPopLoc service four of these requests.

G. Number of Requests

As labeled in Fig. 10, the number of serviced requests increases with the number of requests. For example, with the same resources, MILP services 19 requests when there are 20 requests and services 24 requests when there are 30 requests. This is due to different attributes in each request, including location, computation and communication demand, and CPU frequency, more requests increase the chance for finding requests with less resources. For example, when searching for a request to be serviced using available resources, selecting a request among 30 requests leads to better result than selecting a request among 20 requests. However, as shown in Fig. 10, the percentage of serviced requests decreases with the number of requests. The reason is that due to limited number of UAVs, time slots, and resources, including computation,

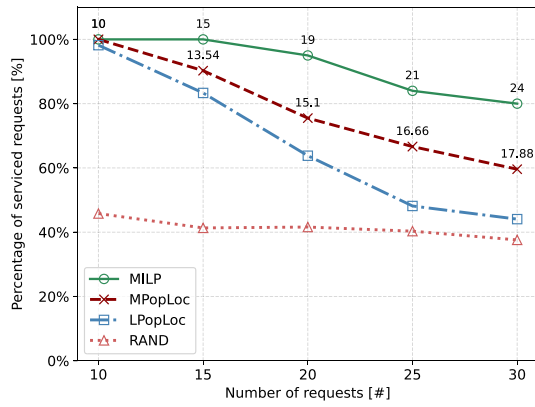


Fig. 10. Percentage of serviced requests by MILP, MPopLoc, LPopLoc, and RAND with different number of requests.

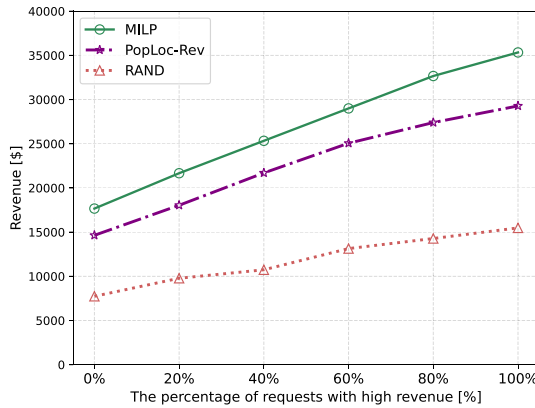


Fig. 11. Total revenue attained by MILP, PopLoc-Rev, and RAND with a different percentage of requests with a high value.

communication, and energy, with an increase in the number of requests, a lower percentage of the requests can be serviced. In MILP, when there are 20 requests, 95% of them are serviced, and when there are 30 requests, 80% of them are serviced.

H. Revenue

This section studies varying percentages of requests with a high value or payment. Specifically, at zero percent, all requests have the same value of \$1000. At 20% and given 20 requests, then four requests will pay the operator \$2000 if serviced; other requests pay \$1000. The evaluation also studies how PopLoc performs when it considers the value of requests. Specifically, instead of sorting requests based on the popularity of their location, we sort them according to their value; this modified algorithm is referred to as PopLoc-Rev.

As depicted in Fig. 11, the revenue received by an operator increases with the percentage of high value requests. RAND performs poorly because requests are randomly selected regardless of their value. However, with more high value requests, it also exhibits an upward trend.

VIII. CONCLUSION

This article has addressed the challenging problem of optimizing the deployment of UAVs to service requests requiring tasks to be executed at various geographical locations,

some of which necessitate communication between them. It has presented the first MILP formulation for the said novel problem, which can be used to determine the optimal assignments of UAVs and VNFs to maximize an operator's revenue. Additionally, it introduced a heuristic method, MPopLoc, which prioritizes requests at popular locations while minimizing UAV travel costs. The results show that MPopLoc achieved 95.14% of the optimal results computed by MILP.

Looking ahead, there are the following possible future directions. One direction involves the exploration of distributed algorithms, enabling decentralized decision making to optimize resource allocation and task assignments. Another direction is accommodating dynamic service requests scenarios, where requests arrive unpredictably over time. In this respect, a future work can center on developing adaptive algorithms and strategies to dynamically assign UAVs to incoming requests.

REFERENCES

- [1] L. Yang, J. Fan, Y. Liu, E. Li, J. Peng, and Z. Liang, "A review on state-of-the-art power line inspection techniques," *IEEE Trans. Instrum. Meas.*, vol. 69, no. 12, pp. 9350–9365, Dec. 2020.
- [2] N. Nomikos, P. K. Gkonis, P. S. Bithas, and P. Trakadas, "A survey on UAV-aided maritime communications: Deployment considerations, applications, and future challenges," *IEEE Open J. Commun. Soc.*, vol. 4, pp. 56–78, 2023.
- [3] P. Boccadoro, D. Striccoli, and L. A. Grieco, "An extensive survey on the Internet of Drones," *Ad Hoc Netw.*, vol. 122, Nov. 2021, Art. no. 102600. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1570870521001335>
- [4] M. Gharibi, R. Boutaba, and S. L. Waslander, "Internet of Drones," *IEEE Access*, vol. 4, pp. 1148–1162, 2016.
- [5] A. Alcántara, J. Capitán, R. Cunha, and A. Ollero, "Optimal trajectory planning for cinematography with multiple unmanned aerial vehicles," *Robot. Auton. Syst.*, vol. 140, Jun. 2021, Art. no. 103778.
- [6] W. Feng, L. Dandan, and C. Qiang, "Discussion on the application of UAV oblique photography in the registration of rural housing and real estate integration," in *Proc. IEEE 5th Int. Conf. Intell. Transp. Eng.*, Beijing, China, 2020, pp. 290–295.
- [7] "Docker: Accelerated container application development." Accessed: Nov. 2023. [Online]. Available: <https://www.docker.com>
- [8] A. M. Potdar, D. Narayan, S. Kengond, and M. M. Mulla, "Performance evaluation of docker container and virtual machine," *Procedia Comput. Sci.*, vol. 171, pp. 1419–1428, Jun. 2020.
- [9] C. Delgado, J. R. Gállego, M. Canales, J. Ortin, S. Bousnina, and M. Cesana, "An optimization framework for resource allocation in virtual sensor networks," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, San Diego, CA, USA, 2015, pp. 1–7.
- [10] H. Afifi, F. J. Sauer, and H. Karl, "Reinforcement learning for admission control in wireless virtual network embedding," in *Proc. IEEE Int. Conf. Adv. Netw. Telecomm. Syst. (ANTS)*, 2021, pp. 84–89.
- [11] A. Roy, S. Misra, S. Bandyopadhyay, and A. Jamalipour, "Mobile sensor-cloud for rendering sensors-as-a-service," *IEEE Syst. J.*, vol. 15, no. 4, pp. 5174–5185, Dec. 2021.
- [12] A. Roy, S. Misra, and S. Nag, "PRIME: An optimal pricing scheme for mobile sensors-as-a-service," *IEEE Trans. Mobile Comput.*, vol. 21, no. 4, pp. 1362–1373, Apr. 2022.
- [13] S. Abdelwahab, B. Hamdaoui, M. Guizani, and T. Znati, "Efficient virtual network embedding with backtrack avoidance for dynamic wireless networks," *IEEE Trans. Wireless Commun.*, vol. 15, no. 4, pp. 2669–2683, Apr. 2016.
- [14] W. Fan et al., "Dynamic virtual network embedding of mobile cloud system based on global resources in Internet of Vehicles," *IEEE Trans. Veh. Technol.*, vol. 70, no. 8, pp. 8161–8174, Aug. 2021.
- [15] J. Miao, N. Lv, K. Chen, W. Gao, and Y. Zhang, "A reliable interference-aware mapping algorithm for airborne tactical network virtualization," *IEEE Access*, vol. 9, pp. 5083–5096, 2021.
- [16] M. Alharthi, A.-E. M. Taha, and H. S. Hassanein, "Evaluating softwarization gains in drone networks," in *Proc. IEEE Global Commun. Conf.*, Madrid, Spain, 2021, pp. 1–6.

- [17] M. Alharthi, A.-E. M. Taha, and H. S. Hassanein, "Utilizing network function virtualization for drone-based networks," in *Proc. IEEE Global Commun. Conf.*, Taipei, Taiwan, 2020, pp. 1–5.
- [18] K. Wu, K.-W. Chin, and S. Soh, "Multi-UAVs network design algorithms for computed rate maximization," *IEEE Trans. Mobile Comput.*, early access, Jan. 18, 2024, doi: [10.1109/TMC.2024.3355391](https://doi.org/10.1109/TMC.2024.3355391).
- [19] A. A. Khuwaja, Y. Chen, N. Zhao, M.-S. Alouini, and P. Dobbins, "A survey of channel modeling for UAV communications," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 4, pp. 2804–2821, 4th Quart., 2018.
- [20] Q. Wang, M. Hempstead, and W. Yang, "A realistic power consumption model for wireless sensor network devices," in *Proc. 3rd Annu. IEEE Commun. Soc. Sensor Ad Hoc Commun. Netw.*, Reston, VA, USA, 2006, pp. 286–295.
- [21] Y. Zeng, J. Xu, and R. Zhang, "Energy minimization for wireless communication with rotary-wing UAV," *IEEE Trans. Wireless Commun.*, vol. 18, no. 4, pp. 2329–2345, Apr. 2019.
- [22] Y. Wen, W. Zhang, and H. Luo, "Energy-optimal mobile application execution: Taming resource-poor mobile devices with cloud clones," in *Proc. IEEE INFOCOM*, Orlando, FL, USA, 2012, pp. 2716–2720.
- [23] A. P. Miettinen and J. K. Nurminen, "Energy efficiency of mobile clients in cloud computing," in *Proc. 2nd USENIX Workshop Hot Topics Cloud Comput.*, Boston, MA, USA, 2010, pp. 1–7.
- [24] R. P. Sokkappa, "The cost-constrained traveling salesman problem," Ph.D. dissertation, Dept. Oper. Res. Comm. Grad. Stud., Stanford Univ., Stanford, CA, USA, 1991.
- [25] "DJI Mavic 3 specifications." Accessed: Mar. 2023. [Online]. Available: <https://www.dji.com/au/mavic-3/specs>
- [26] N. Ahmed, S. S. Kanhere, and S. Jha, "On the importance of link characterization for aerial wireless sensor networks," *IEEE Commun. Mag.*, vol. 54, no. 5, pp. 52–57, May 2016.
- [27] W. Yu, A. Najafi, Y. Huang, and A. Garcia-Ortiz, "Combination of task allocation and approximate computing for fog-architecture-based IoT," *IEEE Internet Things J.*, vol. 8, no. 9, pp. 7638–7648, May 2021.
- [28] "Smart controller hardware." Accessed: Mar. 2023. [Online]. Available: <https://forum.dji.com/thread-178246-1-1.html>



Athena Forghani received the B.Sc. and M.Sc. degrees in applied mathematics from Islamic Azad University Isfahan (Khorasgan) Branch, Isfahan, Iran, in 2008 and 2011, respectively, and the Ph.D. degree in industrial engineering from Islamic Azad University, Science and Research Branch, Tehran, Iran, in 2019. She is currently pursuing the Ph.D. degree in telecommunications engineering with the University of Wollongong, Wollongong, NSW, Australia.

Her research interests include optimization and Internet of Things.



Kwan-Wu Chin received the Bachelor of Science degree (with First-Class Hons.) and the Ph.D. degree (with commendation) from Curtin University, Perth, WA, Australia, in 1997 and 2000, respectively.

He is an Associate Professor with the University of Wollongong, Wollongong, NSW, Australia. He then joined Motorola, Chicago, IL, USA, as a Senior Research Engineer. In 2004, he joined the University of Wollongong as a Senior Lecturer and was promoted to an Associate Professor in 2011. To date, he holds four U.S. patents, and has published

more than 200 conference and journal articles. His current research areas include medium access control protocols for wireless networks, and resource allocation algorithms/policies for communications networks.



Montserrat Ros (Senior Member, IEEE) received the B.E. degree in computer systems/B.Sc. degree in mathematics (with First-Class Hons.) and the Ph.D. degree in computer engineering from the University of Queensland, Brisbane, QLD, Australia, in 2000 and 2007, respectively.

She is an Associate Professor with the School of Electrical, Computer and Telecommunications Engineering and an Associate Dean Education with the Faculty of Engineering and Information Sciences, University of Wollongong, Wollongong, NSW, Australia. She has published over 80 peer-reviewed papers. Her research interests include embedded systems, Internet of Things, sensor networks data fusion, code compression, and engineering education.

Dr. Ros was awarded the 2019 Citation for Outstanding Contributions to Student Learning at the Australian Awards for University Teaching and was named a UOW 2016 Woman of Impact for inspiring the young STEM generation.